

```
title: "HarvardX Data Science Capstone-MOVIELENS" author: "Jorge Castro Martínez" date:
"2024-22-07" output: tufte::tufte_html: toc: yes tufte::tufte_handout: citation_package:
natbib latex_engine: xelatex tufte::tufte_book: citation_package: natbib latex_engine: xelatex
link-citations: yes always_allow_html: true — {r_setup, include=FALSE}
knitr::opts_chunk$set(warning = FALSE,message = FALSE,echo = TRUE)
```

Introduction

#This project represents the culmination of the Harvardx Data Science certificate program, which involves a comprehensive analysis of the Movielens database, a vast repository of user ratings and movie attributes. The database comprises approximately 10 million data points related to 70,000 distinct users and 11,000 movies, divided into training and test sets. The project is divided into two primary components: firstly into an exploratory data analysis phase and secondly it is based on the technique of machine learning model development utilizing the K-Nearest Neighbor (KNN) algorithm and multiple linear regression.

Methodology

#The project commences with an in-depth exploratory data analysis where the database is scrutinized and transformed to gain a deeper understanding of the data distribution, patterns, and correlations. Summary statistics and visualizations are employed to elucidate the characteristics of the data. Subsequently, the KNN algorithm is selected and implemented as a machine learning model to develop a movie recommendation system leveraging the Movielens datasets. The algorithm estimates the density function of the predictor variable for each class, enabling non-parametric supervised learning and classification based on proximity.

#In addition, multiple linear regression is developed as a versatile statistical model to evaluate the relationships between a continuous destination and the predictors.

#The ultimate objective is to develop a prediction model that can accurately forecast movie ratings based on user preferences, facilitating genre-based recommendations. The performance of the developed model is evaluated using the Root Mean Square Error (RMSE) metric, comparing the predictions with the predefined final_holdout_test algorithm

First part: Exploratory data analysis -EDA

The complete database is considered for all 193 countries

```
# If the output is html, the code is included
#knitr::opts_chunk$set(echo = knitr::is_html_output(),
fig.align='center')
```

```
# Create edx set, validation set (final hold-out test set)
```

```

```{r}
library(tidyverse)
library(caret)
library(data.table)

Download and preprocess data (same as before)

Create a temporary file for the data set
dl <- tempfile()

Download MovieLens Dataset
download.file("http://files.grouplens.org/datasets/movielens/ml-10m.zip", dl)

Read grade data and replace ':::' with tabs for easier processing
ratings <- fread(text = gsub(":::", "\t", readLines(unzip(dl, "ml-10M100K/ratings.dat"))),
 col.names = c("userId", "movieId", "rating",
 "timestamp"))

Read the movie data and split each line by ':::'
movies <- str_split_fixed(readLines(unzip(dl, "ml-10M100K/movies.dat")), "\\:::", 3)
colnames(movies) <- c("movieId", "title", "genres")

Convert movies to a data frame and change column types
movies <- as.data.frame(movies) %>%
 mutate(movieId = as.numeric(movieId),
 title = as.character(title),
 genres = as.character(genres))

Join ratings and movie data into a single data frame
movielens <- left_join(ratings, movies, by = "movieId")

Split data into training (edx) and validation (final_holdout_test) sets
set.seed(1)
test_index <- createDataPartition(y = movielens$rating, times = 1, p = 0.1, list = FALSE)
edx <- movielens[-test_index,]
temp <- movielens[test_index,]

Make sure the userId and movieId in the validation set are also in the training set
validation <- temp %>%
 semi_join(edx, by = "movieId") %>%
 semi_join(edx, by = "userId")

Add rows removed from the validation set back to the training set

```

```

removed <- anti_join(temp, validation)
edx <- rbind(edx, removed)

Clean up the workspace by removing temporary objects
rm(dl, ratings, movies, test_index, temp, movielens, removed)

#Loading packages
#install.packages("devtools")

#Loading libraries
#library(dplyr)
library(tidyverse)
#library(tidyr)
#library(stringr)
#library(forcats)
#library(ggplot2)
#library(gt)
#library(DataExplorer)
#install.packages("devtools")
#library(wordcloud2)
#library(stringr)
#install.packages("LaTeX")
#install.packages("webshot")
#webshot::install_phantomjs()

```

## First part: Exploratory data analysis -EDA

```

#Data frame analysis

edx
validation
head(edx,5)
head(validation,5)
glance<-glimpse(c(edx,validation))
names(edx)
names(validation)
summary(edx$rating)
summary(edx$movieId)
Genres <-unique(edx$genres)
Genres

```

## Count of null records in the database

```

sum(is.na(c(edx,validation)))

```

## Delete rows with missing values

```
edx <- na.omit(edx)
validation <- na.omit(validation)
```

## verify that the nulls were removed

```
sum(is.na(edx))
sum(is.na(validation))
str(edx)
str(validation)
```

#Both have the same number of variables:

```
#View(edx)
#View(validation)
```

```
dim(edx)
dim(validation)
```

```
#Variables contained in the database.
#userId** (the identification number for each user)
#movieId** (the identification number for each movie),
#rating** (the rating of a movie by a user),
#timestamp** (the timestamp of the rating provided by a user),
#title** (the title of each movie including the year of release),
#genres** (a list of genres for each movie).
#date(dates for each observation)
#rating year and ratingmoth
```

## edx and validation dataset

```
edx %>% summarize(Users = n_distinct(userId),
 Movies = n_distinct(movieId))
edx
```

#Convert character data to date and time.

```
edx$date <- as.POSIXct(edx$timestamp, origin="2005-01-01")
```

```
validation$date <- as.POSIXct(validation$timestamp, origin="1970-01-01")
```

```
#Column Check
Datetime<-str(c(edx$date,validation$date))
```

# Extract the rating year and rating month in both datasets from the converted timestamp

```
edx$ratingYear <- format(edx$date,"%Y")
edx$ratingMonth <- format(edx$date,"%m")
```

```
validation$ratingYear <- format(validation$date,"%Y")
validation$ratingMonth <- format(validation$date,"%m")
```

```
#View(edx)
#View(validation)
str(edx)
```

### Convert the new columns in numeric data type

```
edx$ratingYear <- as.numeric(edx$ratingYear)
edx$ratingMonth <- as.numeric(edx$ratingMonth)
```

```
validation$ratingYear <- as.numeric(validation$ratingYear)
validation$ratingMonth <- as.numeric(validation$ratingMonth)
```

```
str(edx)
```

```
#Movies in the different genres
```

```
Moviesbygenres <- edx %>%
 group_by(genres) %>%
 summarize(Total = n())
```

```
Moviesbygenres
```

### Filter the year and genres for each movie in the edx dataset related with drama and romance

```
Moviesgenreby2005 <- edx %>%
 filter(ratingYear == 2005 & genres %in% c("Drama","Romance"))
```

```
Moviesgenreby2005
```

### Filter the genres for each movie in the validation dataset related with drama and romance in august month

```
edx %>%
 filter(ratingMonth == "08" & genres %in% c("Drama", "Comedy"))
```

```
edx
```

```
#Distribution of ratings by geo (histogram and violin)
```

```
ggplot(edx, aes(x = rating)) +
 geom_histogram(binwidth = 0.9, fill = "lightblue", color = "red") +
```

```

labs(title = "Distribution of ratings")

ggplot(edx, aes(x = 1, y = rating)) +
 geom_violin(color="black",fill="blue") +
 labs(title = "Violin Plot to show rating distribution", y =
"rating")

```

## Distribution of the rating frequency through the years

```

hist(edx$ratingYear, main="Distribution of the rating frequency
through the years",
 xlab="Years",
 ylab="Rating frequency", col="purple")

#Word clouds

library(wordcloud2)
library(stringr)

tag_data <- edx$title
tag_data <- as.data.frame(tag_data)

word_freq <- table(tag_data$tag)
word_freq <- as.data.frame(word_freq)

wordcloud2(word_freq, size = 1, minRotation = -0.52, maxRotation = -
0.52, rotateRatio = 2)

library(stringr) # Definir la variable objetivo (genres)

genres <- c("Action", "Adventure", "Animation",
 "Children", "Comedy", "Crime",
 "Documentary", "Drama", "Fantasy",
 "Film-Noir", "Horror", "Musical",
 "Mystery", "Romance", "Sci-Fi",
 "Thriller", "War", "Western")

genres_df <- data.frame(
 Genres = genres,
 Count = sapply(genres, function(x) {
 sum(str_detect(edx$genres, x))
 })
)

print(genres_df)

#As dataframe

```

```
training_set <- data.frame(edx)
testing_set <- data.frame(validation)

str(training_set)
str(testing_set)
```

### Average numerical variables

```
training_set <- training_set %>%
 group_by(genres) %>%
 summarize(across(where(is.numeric), mean))

str(training_set)

testing_set <- testing_set %>%
 group_by(genres) %>%
 summarize(across(where(is.numeric), mean))

str(testing_set)
```

### Delete some columns`

```
training_set<-training_set[,-c(5:7)]
training_set

testing_set<-testing_set[,-c(5:7)]
testing_set
```

## K-Nearest Neighbors (KNN) classification

### define the distance between all observations based on the features.

#It is a simple algorithm to understand and implement, and can be used for classification tasks. #This provides Interpretable results that can be visualized and understood as the predicted class is based on the labels of the nearest neighbors in the training data.

```
library(class)
set.seed(123) # Set random seed for reproducibility
```

```
k <- 5 # This number of neighbors is considered optimal to execute
the logarithm
```

### Convert Status variable to a factor

```
training_set$genres <- factor(training_set$genres, levels =
c("Action", "Adventure", "Animation", "Children", "Comedy", "Crime",
```

```
"Documentary", "Drama", "Fantasy","Film-Noir", "Horror",
"Musical","Mystery", "Romance", "Sci-Fi","Thriller", "War",
"Western"))
```

```
str(training_set)
```

```
testing_set$genres <- factor(testing_set$genres, levels = c("Action",
"Adventure", "Animation","Children", "Comedy", "Crime",
```

```
"Documentary", "Drama", "Fantasy","Film-Noir", "Horror",
"Musical","Mystery", "Romance", "Sci-Fi","Thriller", "War",
"Western"))
```

```
str(testing_set)
```

## clasificación KNN

**It seeks to predict the genre of the movies, define the distance between all observations based on the features.**

#It is a simple algorithm to understand and implement, and can be used for classification tasks. #This provides Interpretable results that can be visualized and understood as the predicted class is based on the labels of the nearest neighbors in the training data.

### Delete rows with missing values

```
training_set <- na.omit(training_set)
testing_set <- na.omit(testing_set)
```

```
y_pred <- knn(train = training_set[, -1],
 test = testing_set[, -1],
 cl = training_set$genres,
 k = k)
```

```
y_pred
plot(y_pred)
```

## Most of the neighbors belong to Comedy and Horror

## Evaluate model performance (e.g. accuracy, F1-score)

```
#library(caret)
confusion_matrix <- confusionMatrix(testing_set$genres, y_pred)
print(confusion_matrix)
```

## Calculate and show precision

#To calculate the accuracy of the KNN algorithm, the two data sets (edx and validation) were taken as reference, implementing the KNN algorithm, #comparing the predicted labels with the real labels of the test set and calculating the accuracy as the proportion of instances correctly classified.

```
accuracy <- mean(testing_set$genres == y_pred)
print(paste("Accuracy:", accuracy))
```

```
library(Metrics)
calcular RMSE Modelo 2
```

```
#RMSE <- sqrt(mean((testing_set$genres - y_pred)^2))
```

#Based on accuracy, similar distances are evident. Overfitting imply that the model is well on the training data but has poor performance when new data is coming.

#Linear Regression

```
training_set2 <- training_set[,2:4]
testing_set2 <- testing_set[,2:4]
```

```
str(training_set2)
```

## Fit the Multiple Linear Regression Model with the Training Set

```
regression_model = lm(formula = rating ~ userId + movieId,
 data = training_set2)
```

## Model Summary

```
summary(regression_model)
```

## Make predictions on the test set

```
predictions <- predict(regression_model, newdata = testing_set2, type
= "response")
```

```
predictions
```

## The RMSE function that will be used in this project is:

```
RMSE2 <- function(true_ratings = testing_set2, predicted_ratings =
training_set2) { sqrt(mean((testing_set2 - training_set2)^2))
}
```

```
library(Metrics)
Calculate the average of movies
mean_training <- mean(training_set2$rating)
mean_testing <- mean(testing_set2$rating)
```

```
RMSE <- sqrt((mean_testing - mean_training))
RMSE
```

```
newpage
```

## Conclusion

#The project's execution and application of various codes in both the exploratory and machine learning stages demonstrate a comprehensive understanding of the data science methods and models covered throughout the certificate program. The comparison of the training and test datasets and the predefined algorithm facilitates model evaluation, highlighting the importance of selecting the appropriate algorithm, whether it be SVN, multiple regression, or component analysis. Moreover, the movieId variable and rating emerge as superior predictors compared to other variables, underscoring their significance in developing an effective movie recommendation system. `